

OE-EC803A — Internet of Things

Semester VIII • 3L:0T:0P • 3 Credits

Module 5: Prototyping Online Components & Embedded Programming

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Prototyping Online Components (APIs)	2
1.1	RESTful APIs	2
1.2	Writing a New API (Backend Endpoint)	2
1.3	Real-Time Reactions: Polling vs. WebSockets	3
1.4	Other Messaging Protocols	4
2	Techniques for Writing Embedded Code	4
2.1	Memory Management & Heap Fragmentation	4
2.2	Power Management and Battery Life Optimization	4
2.3	Embedded Debugging Techniques	6

1. Prototyping Online Components (APIs)

To connect physical hardware to cloud services, devices communicate with web application interfaces called APIs (Application Programming Interfaces).

1.1 RESTful APIs

REST API

A **REST (Representational State Transfer) API** is an architectural style for network applications that uses standard HTTP requests to manage resources. Each resource is identified by a URL (e.g. `'/devices/lamp1'`) and manipulated using standard HTTP verbs.

The primary HTTP verbs used in IoT APIs include:

- **GET:** Retrieve the status/value of a sensor or actuator.
- **POST:** Create a new status log or register a new device.
- **PUT / PATCH:** Update configuration properties or toggle state.
- **DELETE:** Remove a device instance from the registry database.

Common HTTP response status codes returned by IoT APIs:

- **200 OK:** Request succeeded (e.g. GET returned sensor reading).
- **201 Created:** Resource successfully created (e.g. POST registered a new device).
- **400 Bad Request:** Invalid syntax or malformed JSON payload.
- **401 Unauthorized:** Missing or invalid API key / authentication token.
- **404 Not Found:** Target device or resource URL does not exist.
- **500 Internal Server Error:** Server-side runtime database/connection failure.

Data is commonly formatted in **JSON (JavaScript Object Notation)** due to its low parsing overhead and human-readable, key-value structure:

```
{
  "device_id": "sensor_node_04",
  "temperature": 26.5,
  "status": "active"
}
```

1.2 Writing a New API (Backend Endpoint)

Writing a new REST API endpoint involves setting up a server runtime (e.g. Node.js with Express or Python with Flask) that accepts incoming JSON payloads, processes them, and returns standard HTTP response codes.

Express.js IoT Server-Side Logging API

Below is a standard server-side Node.js script that listens for HTTP `POST` requests from IoT sensor nodes and saves the reading:

```
const express = require('express');
const app = express();
app.use(express.json()); // Parse JSON payloads

app.post('/api/telemetry', (req, res) => {
  const { device_id, temperature } = req.body;

  if (!device_id || temperature === undefined) {
    return res.status(400).json({ error: "Missing parameters" });
  }

  console.log(`Log from ${device_id}: ${temperature} C`);
  // (In production: Insert data into MongoDB or PostgreSQL database)

  res.status(201).json({
    message: "Data logged successfully",
    timestamp: new Date()
  });
});

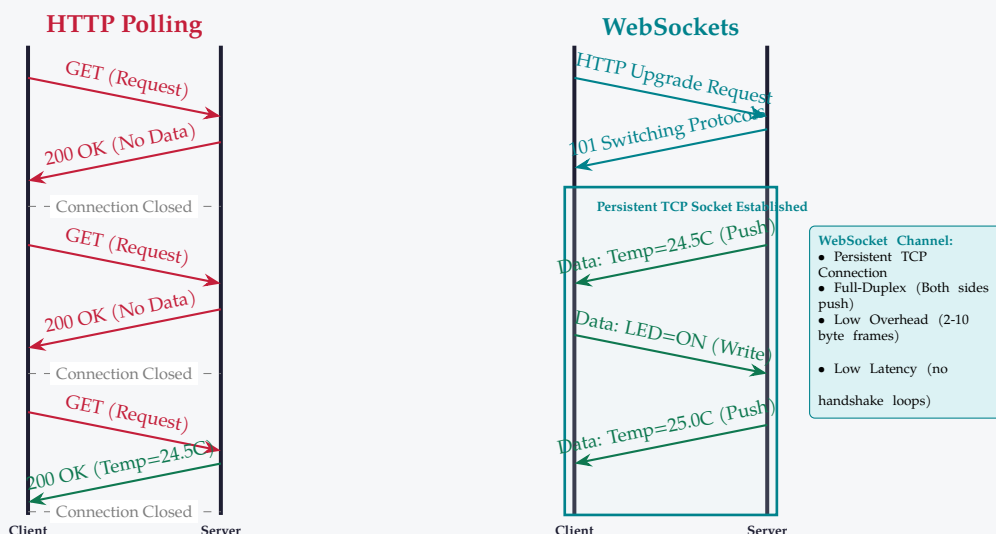
app.listen(3000, () => console.log('IoT Server running on port 3000'));
```

1.3 Real-Time Reactions: Polling vs. WebSockets

IoT systems need to push real-time events (e.g., triggering an alarm).

- **HTTP Polling:** The client periodically sends GET requests to the server (e.g., every 5 seconds) to check for updates. This generates significant network overhead and latency.
- **HTTP Long Polling:** The server holds the client's request open until new data is available, responding immediately, then closing the connection.
- **WebSockets:** Establishes a persistent, bi-directional TCP connection. Both client and server can send data packets at any time with minimal header overhead, enabling sub-millisecond real-time reactions.

HTTP Polling vs. WebSockets Communication Flow



1.4 Other Messaging Protocols

- **XMPP (Extensible Messaging and Presence Protocol):** XML-based instant messaging protocol. Heavy overhead but features strong built-in security and routing.
- **AMQP (Advanced Message Queuing Protocol):** An open standard queuing protocol designed for enterprise transaction messaging, providing high reliability.

2. Techniques for Writing Embedded Code

Microcontrollers are resource-constrained devices, requiring specialized programming practices.

2.1 Memory Management & Heap Fragmentation

Heap Fragmentation

Heap Fragmentation occurs when dynamic memory allocation (`'malloc()'` and `'free()'`) allocates and deallocates varying memory chunks over time, leaving small, unusable gaps in the heap. In microcontrollers with only a few kilobytes of RAM, this can eventually prevent large allocations, leading to program crashes.

To prevent fragmentation, embedded code should:

- Avoid dynamic memory (`'new'`, `'malloc'`, `'String'` objects).
- Prefer static global allocation (defining buffer arrays with fixed sizes at compile time) or stack-based local variables.

2.2 Power Management and Battery Life Optimization

Many IoT devices run on batteries, requiring strict power management:

- **Active Mode:** CPU running at full clock speed, radio transmitting. Draws high current

(10mA to 150mA).

- **Light Sleep:** CPU clock is gated/halted, RAM is preserved. Draws microamps ($100\mu\text{A}$ to $500\mu\text{A}$).
- **Deep Sleep:** CPU, RAM, and internal oscillators are completely shut down. Only an ultra-low-power Real-Time Clock (RTC) or external interrupt remains active. Draws nanoamps ($1\mu\text{A}$ to $15\mu\text{A}$). The device wakes up, reads sensors, transmits, and immediately sleeps again.

Duty Cycling & Battery Life

Duty Cycling is the ratio of active time (t_{active}) to the total period ($T = t_{\text{active}} + t_{\text{sleep}}$):

$$D = \frac{t_{\text{active}}}{t_{\text{active}} + t_{\text{sleep}}} \quad (1)$$

The average current draw (I_{avg}) is calculated as:

$$I_{\text{avg}} = D \cdot I_{\text{active}} + (1 - D) \cdot I_{\text{sleep}} \quad (2)$$

Battery lifetime (L in hours) for a battery with capacity C (in mAh) is:

$$L = \frac{C}{I_{\text{avg}}} \quad (3)$$

Numerical Example: IoT Battery Life Calculation

Scenario: An IoT sensor node wakes up once every 10 minutes (600 seconds) for a duration of 2 seconds to read a sensor and transmit it via Wi-Fi.

- Active Current (I_{active}) = 80 mA
- Sleep Current (I_{sleep}) = $15\mu\text{A} = 0.015\text{ mA}$
- Battery Capacity (C) = 2400 mAh (e.g. standard AA cells)

Step 1: Calculate Duty Cycle (D)

$$D = \frac{2\text{ s}}{600\text{ s}} = 0.00333 \quad (0.33\%) \quad (4)$$

Step 2: Calculate Average Current (I_{avg})

$$I_{\text{avg}} = (0.00333 \cdot 80\text{ mA}) + ((1 - 0.00333) \cdot 0.015\text{ mA}) \quad (5)$$

$$I_{\text{avg}} \approx 0.2667\text{ mA} + 0.0150\text{ mA} = 0.2817\text{ mA} \quad (6)$$

Step 3: Calculate Battery Lifetime (L)

$$L = \frac{2400\text{ mAh}}{0.2817\text{ mA}} \approx 8519.7\text{ hours} \approx 355\text{ days} \quad (7)$$

Thus, by employing aggressive duty cycling, a standard battery can sustain the node for approximately one full year despite its high 80mA active transmission current.

2.3 Embedded Debugging Techniques

Debugging on microcontrollers is challenging because they lack screens.

- **Serial Debugging:** Printing variable values to a host computer serial monitor ('Serial.println()'). Simple but introduces delay (timing bugs).
- **In-Circuit Emulation & JTAG:** A hardware interface (Joint Test Action Group) that connects directly to the CPU core, allowing developers to set hardware breakpoints, inspect memory registers, and step through code line-by-line.

Quick Revision — Module 5

Online Components & Embedded Programming | OE-EC803A

Key Formulas and Metrics at a Glance

Duty Cycle	$D = \frac{t_{\text{active}}}{t_{\text{active}} + t_{\text{sleep}}}$ (ratio of active execution time)
Average Current	$I_{\text{avg}} = D \cdot I_{\text{active}} + (1 - D) \cdot I_{\text{sleep}}$ (power profile)
Battery Lifetime	$L = \frac{\text{Battery Capacity (mAh)}}{I_{\text{avg}} \text{ (mA)}}$ (operation hours)

One-Line Definitions

- **REST:** A stateless web architectural style that relies on standard HTTP methods (GET/POST).
- **WebSocket:** A persistent, full-duplex TCP socket connection for real-time web communication.
- **Duty Cycling:** The practice of powering down the device CPU and radio for periodic sleep intervals.
- **Deep Sleep:** The lowest power state of an MCU, turning off all internal blocks except the RTC wakeup timer.
- **JTAG:** A hardware port standard used for direct in-circuit programming, testing, and debugging.

Frequently Asked / Viva Questions

Q1. Why is dynamic memory allocation ('malloc') highly discouraged in microcontroller code?

Microcontrollers have extremely limited RAM (often a few kilobytes) and no virtual memory management. Repeated allocations and deallocations leave small gaps of unused memory on the heap (heap fragmentation). Eventually, a request for a contiguous memory block fails, causing a system crash.

Q2. Compare HTTP Polling and WebSockets for real-time sensor updates.

HTTP Polling requires the client to repeatedly send new request handshakes and headers, creating heavy network congestion and latency. WebSockets establishes a single persistent TCP connection, enabling bi-directional, real-time message streams with minimal packet overhead.

Q3. Calculate the average current and battery life of an IoT node with a 1000mAh battery. It draws 100mA for 2 seconds when active, and 10μA (0.01mA) for 58 seconds during sleep.

Frequently Asked / Viva Questions (contd.)

(1) Total Period $T = 2 + 58 = 60$ seconds. (2) Duty Cycle $D = 2/60 = 0.0333$ (3.33%). (3) Average Current $I_{\text{avg}} = (0.0333 \times 100) + (0.9667 \times 0.01) = 3.33 + 0.0097 \approx 3.34\text{mA}$. (4) Lifetime $L = 1000/3.34 \approx 299.4$ hours (≈ 12.5 days).

Q4. What occurs during deep sleep in an ESP8266 or ESP32 microcontroller?

During deep sleep, the CPU, RAM, and internal Wi-Fi radio are completely powered off. The only part of the chip that remains powered is the low-power RTC timer and select GPIO wake pins. The CPU must re-initialize (reboot) from scratch when waking up.

Q5. What is a REST API HTTP POST request used for in IoT?

A POST request is used to push data from the client to the server, creating a new database record (e.g., posting a new temperature reading log at '/api/v1/telemetry').

Q6. Explain the JTAG debug interface.

JTAG is a hardware debugging protocol. It uses dedicated physical pins on the microcontroller to interface directly with the internal processor tap, allowing developers to halt CPU execution, step through commands, and read register states.

Q7. What is JSON, and why is it preferred over XML in IoT applications?

JSON is a lightweight key-value text format for representing structured data. It has a much smaller character count than XML (no closing tags), which saves bandwidth, and requires significantly less CPU memory and time to parse.

Q8. How does duty cycling extend device battery life?

Sensors rarely need to transmit data continuously. By sleeping for 99% of the time and only waking up for a few seconds to sample and transmit data, the average current draw is reduced to microamps, extending battery life from days to years.

Comparison Snapshot**HTTP Polling vs. WebSockets:**

Feature	HTTP Polling	WebSockets
Connection Type	Temporary; opened and closed per request.	Persistent; remains open until closed.
Communication	Half-duplex (client pull only).	Full-duplex (bi-directional push/pull).
Network Overhead	High (repeats TCP handshakes and HTTP headers).	Minimal (small frame header of 2-10 bytes).
Real-Time Latency	High (depends on polling frequency).	Extremely Low (sub-millisecond).
Best Use Case	Static data dashboards.	Real-time alerts, remote live control.